

雲端計算作業

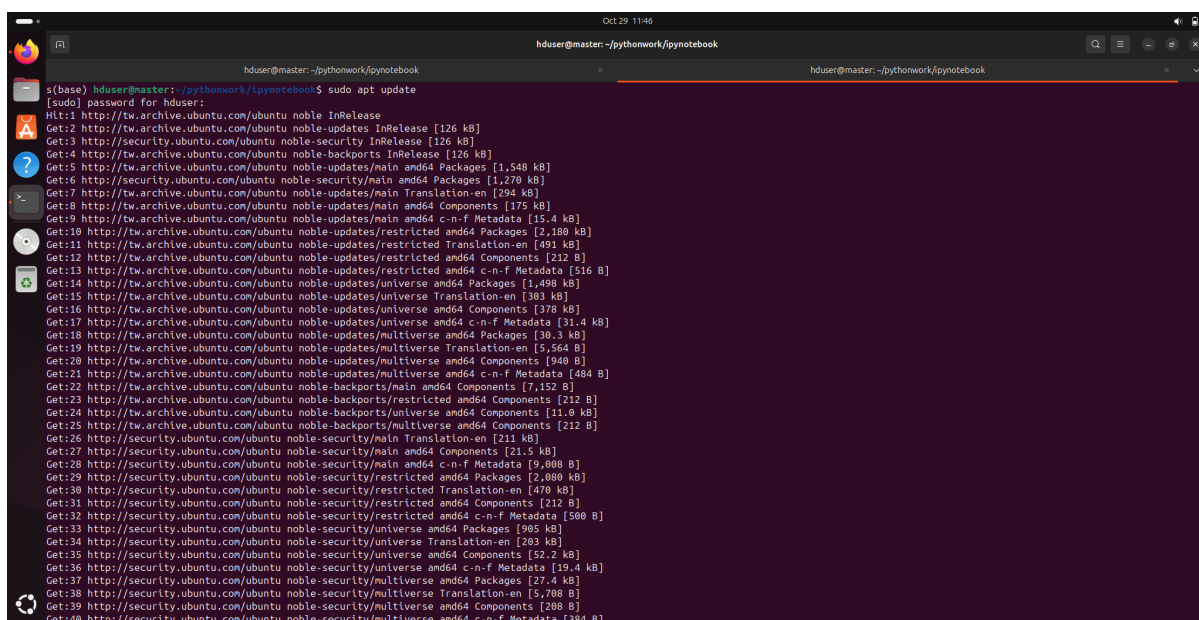
第三章

1. 安裝 guest addition

- 安裝完 Ubuntu 後，開啟 guest addition 會報錯「缺少 bzip2」。
- 打開 Terminal，輸入：

```
sudo apt update  
sudo apt install bzip2 tar
```

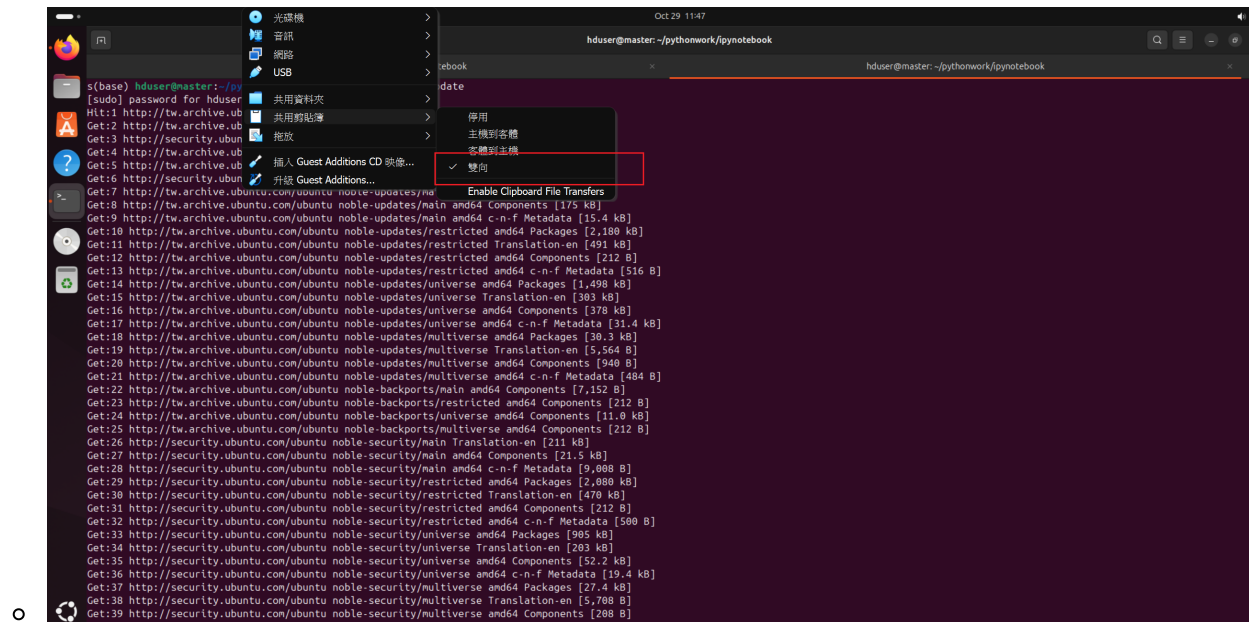
- 安裝完成。



```
Oct 29 11:46  
hduser@master: ~/pythonwork/jupyterbook  
hduser@master: ~/pythonwork/jupyterbook  
hduser@master: ~/pythonwork/jupyterbook  
$ (base) hduser@master: ~/pythonwork/jupyterbook$ sudo apt update  
[sudo] password for hduser:  
Hit:1 http://tw.archive.ubuntu.com/ubuntu noble InRelease  
Get:2 http://tw.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]  
Get:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]  
Get:4 http://tw.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]  
Get:5 http://tw.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,548 kB]  
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,270 kB]  
Get:7 http://tw.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [294 kB]  
Get:8 http://tw.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]  
Get:9 http://tw.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [15.4 kB]  
Get:10 http://tw.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2,180 kB]  
Get:11 http://tw.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [491 kB]  
Get:12 http://tw.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]  
Get:13 http://tw.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 c-n-f Metadata [516 B]  
Get:14 http://tw.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,498 kB]  
Get:15 http://tw.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [303 kB]  
Get:16 http://tw.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [378 kB]  
Get:17 http://tw.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [31.4 kB]  
Get:18 http://tw.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [30.3 kB]  
Get:19 http://tw.archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [5,564 B]  
Get:20 http://tw.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]  
Get:21 http://tw.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 c-n-f Metadata [484 B]  
Get:22 http://tw.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,152 B]  
Get:23 http://tw.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [212 B]  
Get:24 http://tw.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [11.8 kB]  
Get:25 http://tw.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]  
Get:26 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [211 kB]  
Get:27 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]  
Get:28 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [9,088 B]  
Get:29 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2,080 kB]  
Get:30 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [470 kB]  
Get:31 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]  
Get:32 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 c-n-f Metadata [500 B]  
Get:33 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [905 kB]  
Get:34 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [203 kB]  
Get:35 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [52.2 kB]  
Get:36 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [19.4 kB]  
Get:37 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [27.4 kB]  
Get:38 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [5,708 B]  
Get:39 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]  
Get:40 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [384 B]
```

2. 設定雙向剪貼簿

- 關機，設定為雙向剪貼簿。



3. 檢查 Python 版本，安裝 Java

- 先檢查 Python 版本，再決定 Spark 與 Hadoop 版本。
- 檢查指令：

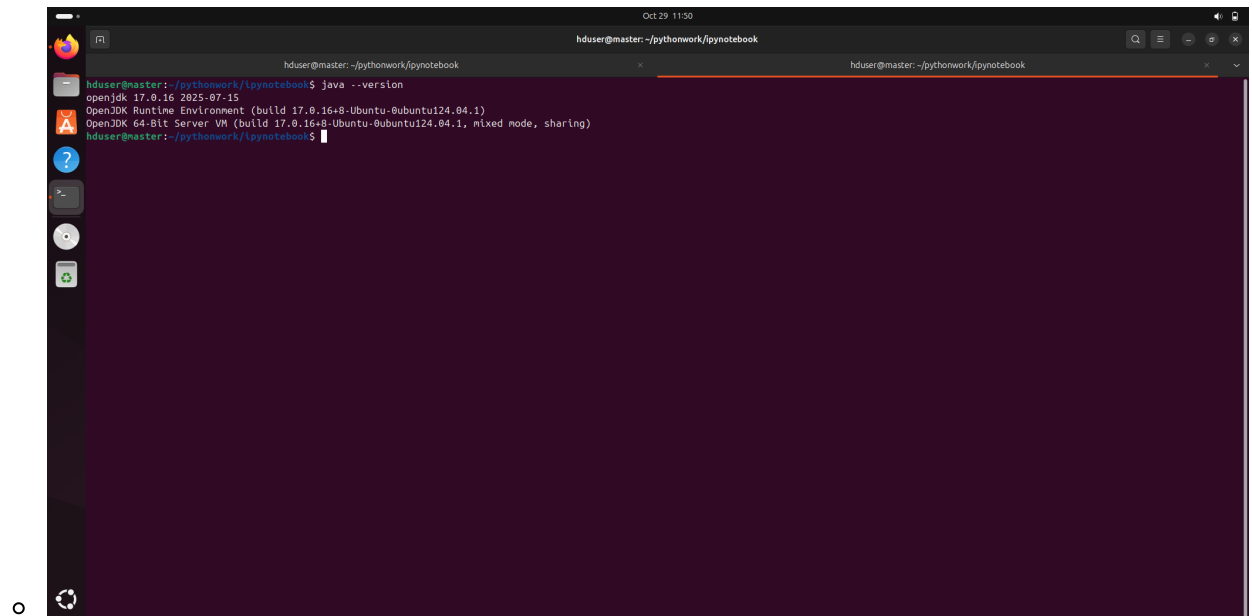
```
python3 -v # 結果顯示 Python 3.12.3
```

第四章

1. 安裝 Java

- 指令：

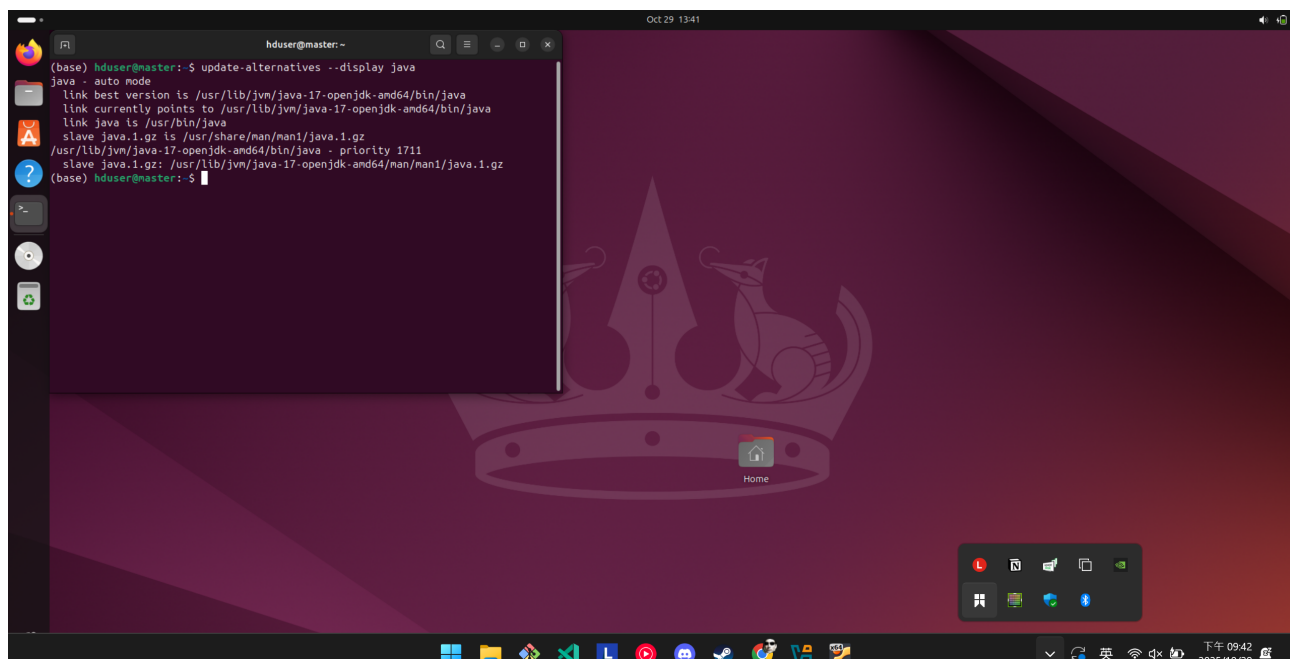
```
java -V  
sudo apt install openjdk-17-jre-headless
```



2. 檢查 Java 安裝位置

- 目的是為了 bashrc 使用
- 指令：

```
update-alternatives --display java
```



3. 安裝 SSH 與 Rsync

- 安裝 SSH：

```
sudo apt-get install ssh
```

- 安裝 Rsync (重複打 SSH 是正確的，系統會提示已安裝)：

```
sudo apt-get install ssh
```

4. 設定 SSH Key

- Ubuntu 24.04 dsa 已棄用，改用 RSA 金鑰：

```
ssh-keygen -t rsa -f ~/.ssh/id_rsa  
ll ~/.ssh  
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
```

5. 安裝 Hadoop (JDK 17 支援 3.3.6)

- 下載並安裝：

```
wget https://dlcdn.apache.org/hadoop/common/hadoop-3.3.6/hadoop-  
3.3.6.tar.gz  
sudo tar -zxvf hadoop-3.3.6.tar.gz  
sudo mv hadoop-3.3.6 /usr/local/hadoop  
ll /usr/local/hadoop
```

6. 安裝 gedit

- 指令：

```
sudo apt install gedit  
sudo apt install gedit-plugins
```

7. 編輯 .bashrc

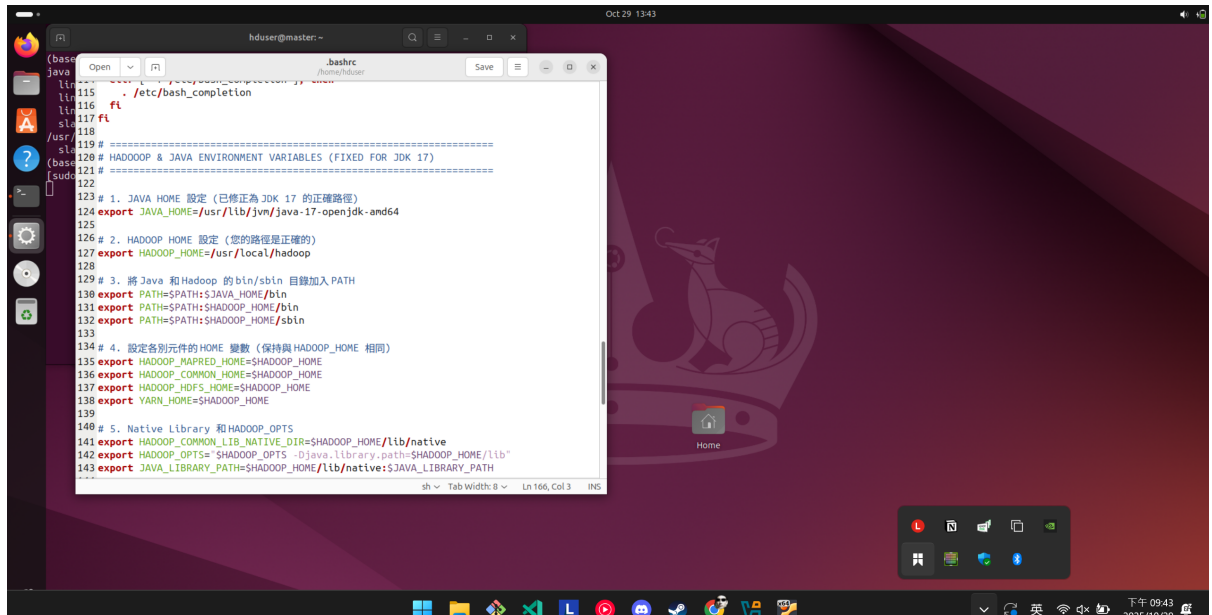
- 指令：

```
sudo gedit ~/.bashrc
```

- 增加以下內容至最底部：

```
# =====  
# HADOOP & JAVA ENVIRONMENT VARIABLES (FIXED FOR JDK 17)  
# =====  
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64  
export HADOOP_HOME=/usr/local/hadoop  
export PATH=$PATH:$JAVA_HOME/bin  
export PATH=$PATH:$HADOOP_HOME/bin
```

```
export PATH=$PATH:$HADOOP_HOME/sbin
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export HADOOP_OPTS="$HADOOP_OPTS -Djava.library.path=$HADOOP_HOME/lib"
export JAVA_LIBRARY_PATH=$HADOOP_HOME/lib/native:$JAVA_LIBRARY_PATH
```



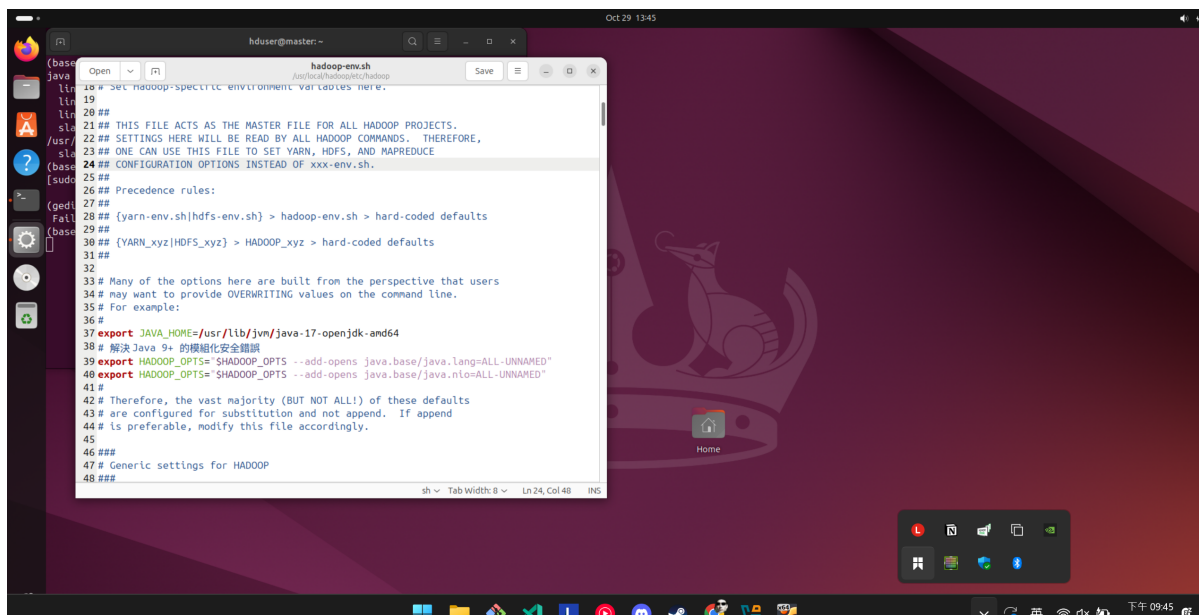
- 儲存並重新載入：

```
source ~/.bashrc
```

8. 修正 Hadoop 配置檔案

- hadoop-env.sh：

```
sudo gedit /usr/local/hadoop/etc/hadoop/hadoop-env.sh
# 搜尋 JAVA_HOME，修正為：
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```



- core-site.xml :

```
sudo gedit /usr/local/hadoop/etc/hadoop/core-site.xml
```

```
<property>
  <name>fs.default.name</name>
  <value>hdfs://localhost:9000</value>
</property>
```

- yarn-site.xml :

```
sudo gedit /usr/local/hadoop/etc/hadoop/yarn-site.xml
```

```
<property>
  <name>yarn.nodemanager.aux-services</name>
  <value>mapreduce_shuffle</value>
</property>
<property>
  <name>yarn.nodemanager.aux-services.mapreduce.shuffle.class</name>
  <value>org.apache.hadoop.mapred.ShuffleHandler</value>
</property>
```

- mapred-site.xml :

```
gedit /usr/local/hadoop/etc/hadoop/mapred-site.xml
```

```
<property>
  <name>mapreduce.framework.name</name>
  <value>yarn</value>
</property>
```

- hdfs-site.xml :

```
sudo gedit /usr/local/hadoop/etc/hadoop/hdfs-site.xml
```

```
<property>
  <name>dfs.replication</name>
  <value>3</value>
</property>
<property>
  <name>dfs.namenode.name.dir</name>
  <value>file:/usr/local/hadoop/hadoop_data/hdfs/namenode</value>
</property>
<property>
  <name>dfs.datanode.data.dir</name>
  <value>file:/usr/local/hadoop/hadoop_data/hdfs/datanode</value>
</property>
```

9. 建立與格式化 HDFS 目錄

- 建立儲存目錄：

```
sudo mkdir -p /usr/local/hadoop/hadoop_data/hdfs/namenode
sudo mkdir -p /usr/local/hadoop/hadoop_data/hdfs/datanode
sudo chown hduser:hduser -R /usr/local/hadoop
hadoop namenode -format
```

- 放寬 Java 權限、解決 Java 9+ 模組化安全錯誤：

```
export HADOOP_OPTS="$HADOOP_OPTS --add-opens java.base/java.lang=ALL-UNNAMED"
export HADOOP_OPTS="$HADOOP_OPTS --add-opens java.base/java.nio=ALL-UNNAMED"
```

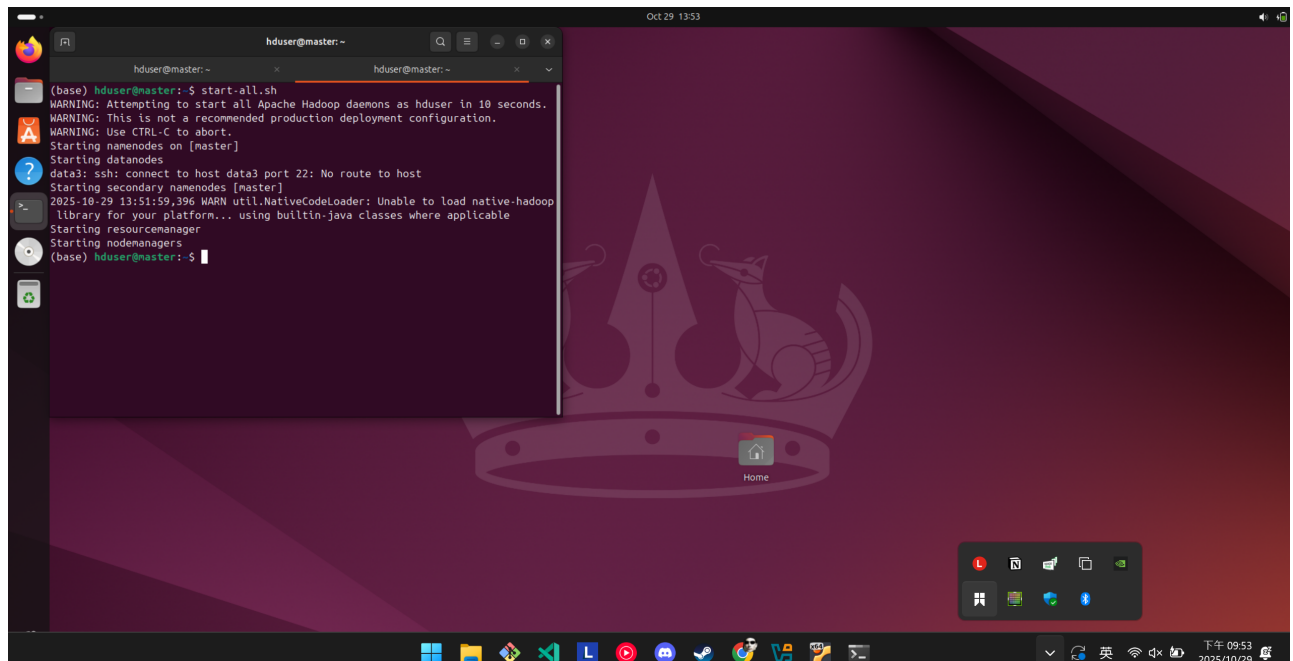
10. 設定 HDFS Web UI port

- 預設為 <http://localhost:9870/> · 改為 <http://localhost:50070>

```
<property>
  <name>dfs.namenode.http-address</name>
  <value>0.0.0.0:50070</value>
  <description>The address and port that the NameNode web UI will
listen on.</description>
</property>
```

11. 啟動 Hadoop

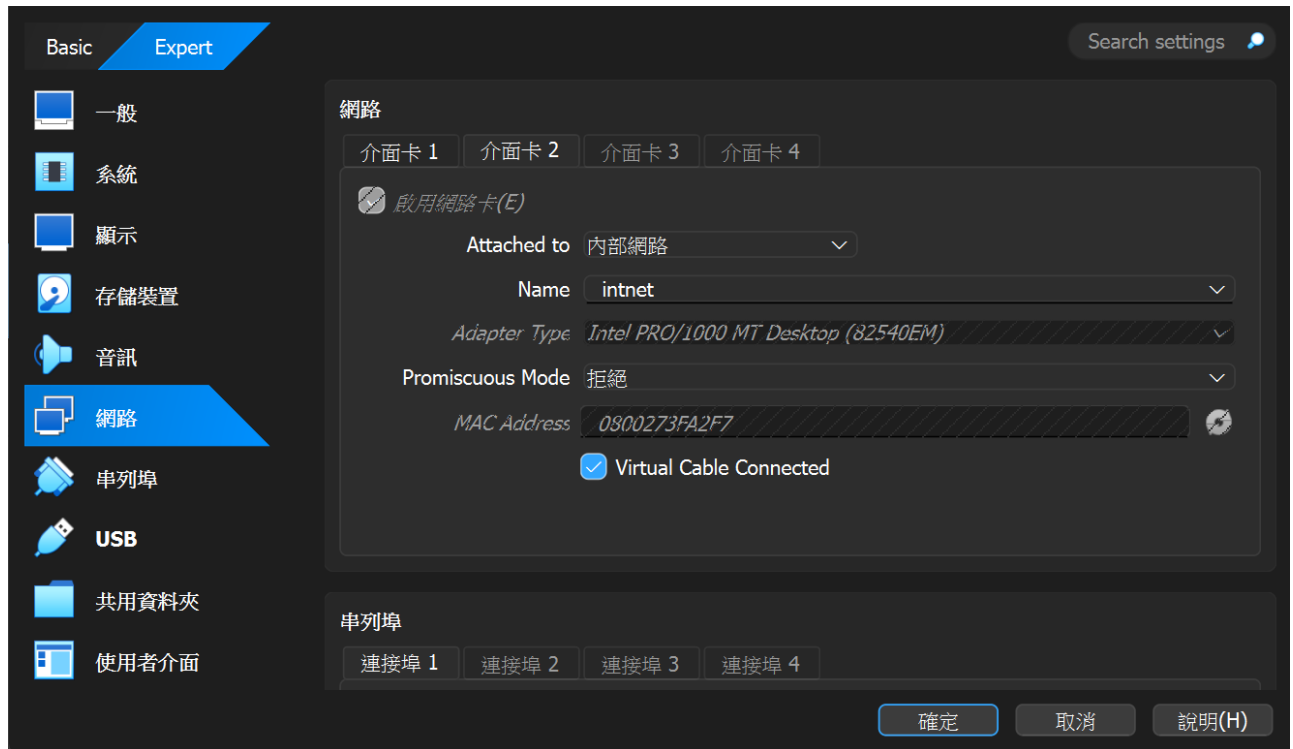
```
start-all.sh
```



第五、六章

1. 虛擬機再製與網路設定

- 關閉虛擬機後，複製 (Clone) 新機，並編輯網路介面卡 2，啟用並設為「內部網路」。



- 啟動 Ubuntu，編輯網路設定檔，採用 Netplan (新版 Ubuntu 已棄用 `/etc/network/interfaces`，且網卡已不再用 `eth0/eth1` 命名)。
- 編輯指令：

```
sudo gedit /etc/netplan/01-network-manager-all.yaml
```

- 範例內容：

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3: # 對外 NAT，用 DHCP
      dhcp4: true
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4] # 強制 Google DNS
    enp0s8: # 內部集群 Host-Only
      dhcp4: no
      addresses: [192.168.56.101/24] # 靜態 IP
```

 更改靜態 ip

- 變更權限：

```
sudo chmod 600 /etc/netplan/01-network-manager-all.yaml
```

- 套用新設定與驗證：

```
sudo netplan apply  
ip a
```

2. 設定 **hostname** 與 **hosts** 清單

- 修改主機名稱：

```
sudo hostnamectl set-hostname data1
```

- 編輯 **hosts** 檔案，新增節點：

```
sudo gedit /etc/hosts
```

```
192.168.56.100 master  
192.168.56.101 data1  
192.168.56.102 data2  
192.168.56.103 data3
```



3. **Hadoop** 組態檔更新

- **core-site.xml**：

```
<property>  
  <name>fs.default.name</name>  
  <value>hdfs://master:9000</value>  
</property>
```

- **yarn-site.xml**：

```
<property>  
  <name>yarn.resourcemanager.resource-tracker.address</name>  
  <value>master:8025</value>  
</property>  
<property>  
  <name>yarn.resourcemanager.scheduler.address</name>  
  <value>master:8030</value>  
</property>
```

```
<property>
  <name>yarn.resourcemanager.address</name>
  <value>master:8050</value>
</property>
<property>
  <name>yarn.nodemanager.aux-services</name>
  <value>mapreduce_shuffle</value>
</property>
<property>
  <name>yarn.nodemanager.aux-services.mapreduce.shuffle.class</name>
  <value>org.apache.hadoop.mapred.ShuffleHandler</value>
</property>
```

- mapred-site.xml :

```
<property>
  <name>mapred.job.tracker</name>
  <value>master:54311</value>
</property>
```

- hdfs-site.xml :

```
<property>
  <name>dfs.replication</name>
  <value>3</value>
</property>
<property>
  <name>dfs.datanode.data.dir</name>
  <value>file:/usr/local/hadoop/hadoop_data/hdfs/datanode</value>
</property>
<property>
  <name>dfs.namenode.http-address</name>
  <value>0.0.0.0:50070</value>
  <description>The address and port that the NameNode web UI will listen on.
</description>
</property>
```

4. 多節點複製及啟動

- 更新 net-tools 與複製設定至 data2、data3、master 節點，並更改 IP/hostname。
- 背景啟動虛擬機 (windows cli) :

```
VBoxManage startvm "data1" --type headless
VBoxManage startvm "data2" --type headless
VBoxManage startvm "data3" --type headless
```

```
命令提示字元
D:\Oracle\VM Folder>cd ..
D:\Oracle>dir
磁碟區 D 中的磁碟沒有標籤。
磁碟區序號: F9A8-C5B9

D:\Oracle 的目錄
2025/09/10 下午 11:37 <DIR> .
2025/09/10 下午 11:36 <DIR> VirtualBox
2025/10/11 下午 07:15 <DIR> VM Folder
                0 個檔案          0 位元組
                3 個目錄    659,539,496,960 位元組可用

D:\Oracle>cd VirtualBox
D:\Oracle\VirtualBox>.\VBoxManage startvm "data1" --type headless
Waiting for VM "data1" to power on...
VM "data1" has been successfully started.

D:\Oracle\VirtualBox>.\VBoxManage startvm "data2" --type headless
Waiting for VM "data2" to power on...
VM "data2" has been successfully started.

D:\Oracle\VirtualBox>.\VBoxManage startvm "data3" --type headless
Waiting for VM "data3" to power on...
VM "data3" has been successfully started.

D:\Oracle\VirtualBox>
```

- 檢查與關機：

```
VBoxManage list runningvms
VBoxManage controlvm "VM名" acpipowerbutton
```

- 測試 SSH、建立 datanode/namenode 目錄，格式化，並啟動 Hadoop multi-node cluster：

```
hadoop namenode -format
start-all.sh
ssh data1 jps
ssh data2 jps
ssh data3 jps
```

5. 多節點 HDFS、YARN，主節點 (master) 與 worker 節點設置

- master 節點 hdfs-site.xml 範例 (additional)：

```
<property>
  <name>dfs.replication</name>
  <value>3</value>
</property>
<property>
  <name>dfs.datanode.data.dir</name>
  <value>file:///usr/local/hadoop/hadoop_data/hdfs/datanode</value>
  <description>DataNode directory for data storage.</description>
</property>
<property>
  <name>dfs.namenode.name.dir</name>
```

```
<value>file:/usr/local/hadoop/hadoop_data/hdfs/namenode</value>
</property>
<property>
  <name>dfs.namenode.http-address</name>
  <value>0.0.0.0:50070</value>
  <description>The address and port that the NameNode web UI will listen on.
</description>
</property>
```

6. 設置 **master/worker** 檔案

- masters :

```
master
```

- workers (新版為 workers · 不是 slaves) :

```
data1
data2
data3
```

7. 利用 **VirtualBox CLI** 管理節點

- Windows VirtualBox CLI 背景啟動資料節點。

8. 建立/移除及權限設定

- 建立 datanode/namenode 目錄 · 並更改目錄權限：

```
sudo rm -rf /usr/local/hadoop/hadoop_data/hdfs
mkdir -p /usr/local/hadoop/hadoop_data/hdfs/datanode
sudo chown -R hduser:hduser /usr/local/hadoop/
```

- 格式化 master node 的 namenode：

```
hadoop namenode -format
```

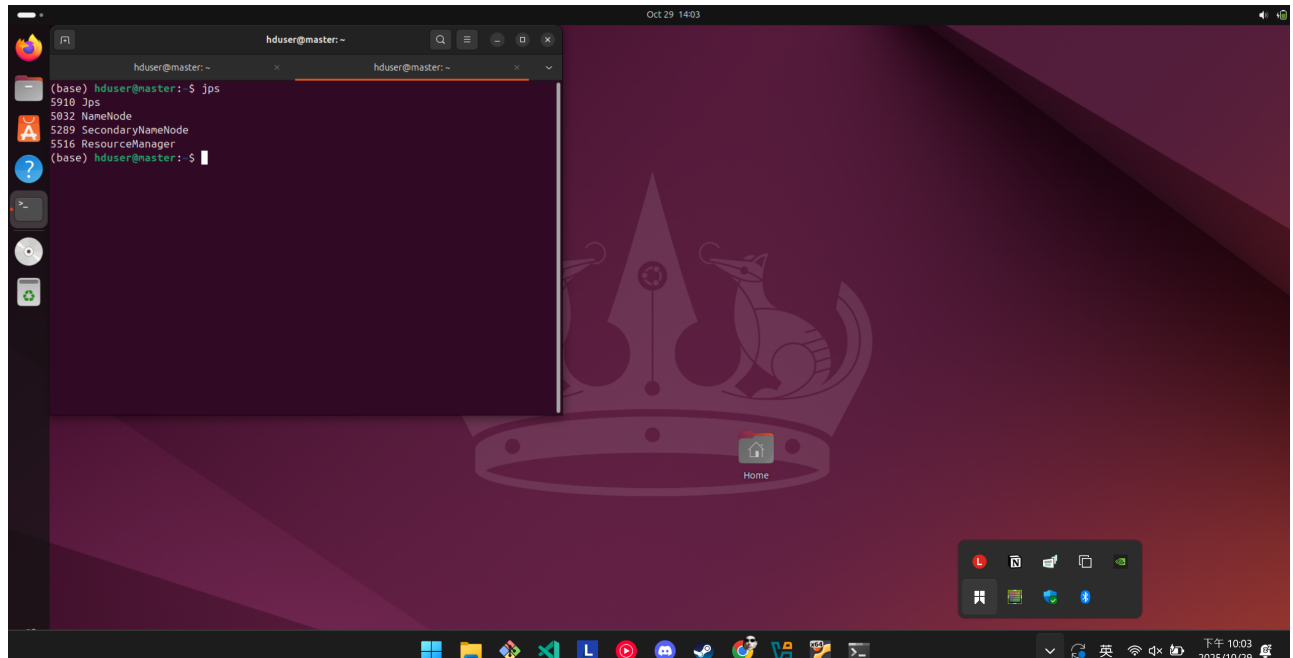
9. 啟動多節點叢集並檢查

- 啟動：

```
start-all.sh
```

- 檢查每個 datanode 是否啟動（連線測試）：

```
ssh data1 jps
ssh data2 jps
ssh data3 jps
```



第八章

1. 安裝 Scala

- 下載 Scala：

```
wget https://www.scala-lang.org/files/archive/scala-2.13.16.tgz
```

- 解壓縮 Scala：

```
tar xvf scala-2.13.16.tgz
```

- 搬移到指定目錄：

```
sudo mv scala-2.13.16 /usr/local/scala
```

- 設定環境參數：

```
sudo gedit ~/.bashrc
export SCALA_HOME=/usr/local/scala
export PATH=$PATH:$SCALA_HOME/bin
```

- 重新載入環境，進入 Scala，測試後離開：

```
source ~/.bashrc
scala          # 若成功進入，輸入 :q 離開
```

2. 安裝 Spark

- 下載 Spark：

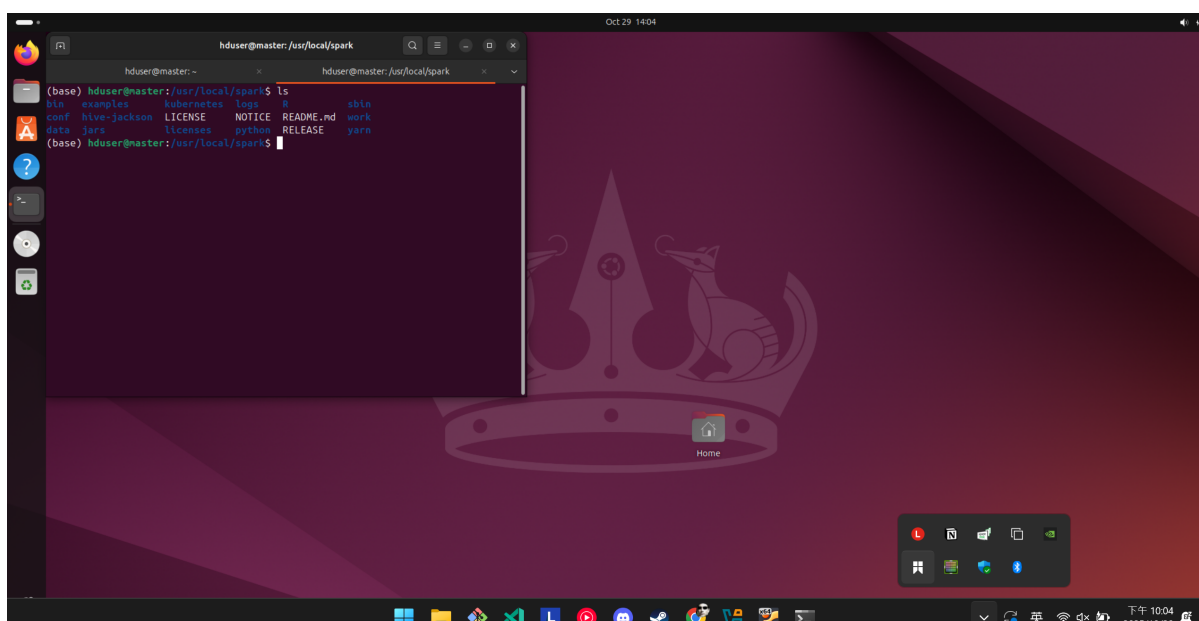
```
wget https://dlcdn.apache.org/spark/spark-4.0.1/spark-4.0.1-bin-hadoop3.tgz
```

- 解壓縮：

```
tar xzf spark-4.0.1-bin-hadoop3.tgz
```

- 搬移到目錄：

```
sudo mv spark-4.0.1-bin-hadoop3 /usr/local/spark/
```



- 設定環境參數：

```
sudo gedit ~/.bashrc
# ===== Spark 4.0 =====
export SPARK_HOME=/usr/local/spark
export PATH=$PATH:$SPARK_HOME/bin
```

- 重新載入：

```
source ~/.bashrc
```

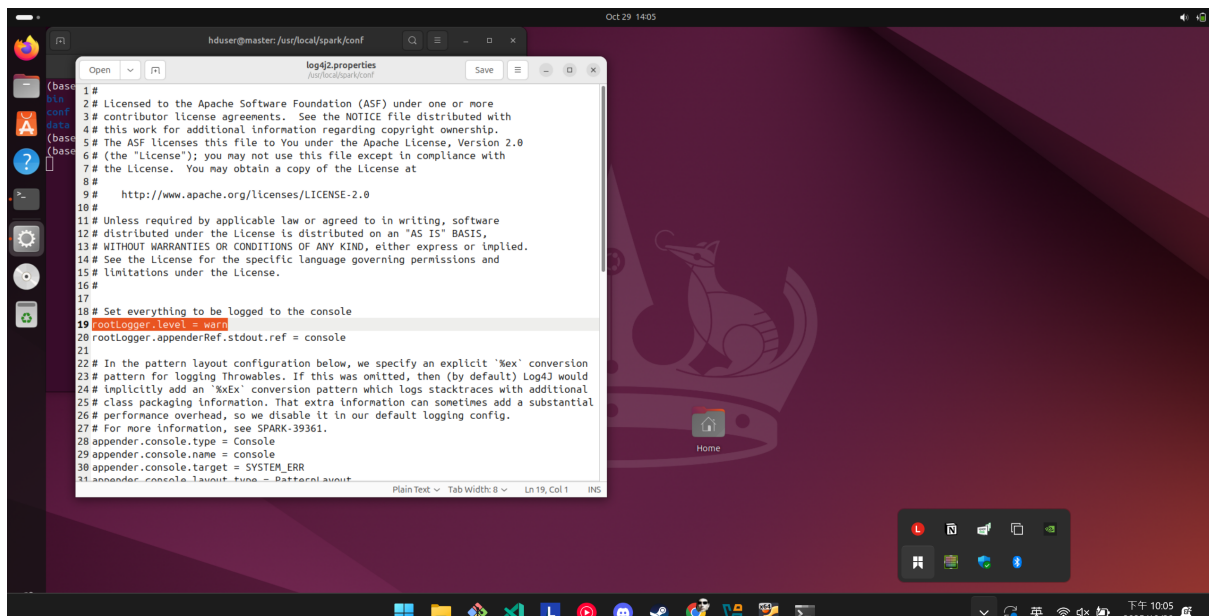
3. Pyspark 啟動與設定 log

- 啟動 pyspark (離開用 `exit()`)：

```
pyspark
```

- 複製 log4j 樣板並編輯 log 等級：

```
cd /usr/local/spark/conf
cp log4j2.properties.template log4j2.properties
sudo gedit log4j2.properties
# 將 rootLogger.level = info 改為 rootLogger.level = warn
```



4. 建立 HDFS 測試目錄與檔案並上傳

- 建立 HDFS user 目錄：

```
hadoop fs -mkdir /user
hadoop fs -mkdir /user/hduser
hadoop fs -mkdir /user/hduser/test
hadoop fs -mkdir -p /usr/hduser/wordcount/input
```

- 建立本機資料夾與檔案：

```
mkdir -p ~/wordcount/input
cp /usr/local/hadoop/LICENSE.txt ~/wordcount/input
ll ~/wordcount/input
```

- 上傳至 HDFS：

```
hadoop fs -copyFromLocal LICENSE.txt /user/hduser/wordcount/input
hadoop fs -ls /user/hduser/wordcount/input
```

5. 執行 Pyspark 測試與讀取資料

- 本機模式啟動：

```
pyspark --master local[*]
```

- 查看模式與讀取本機檔案：

```
sc.master
textFile = sc.textFile("file:/usr/local/spark/README.md")
textFile.count()
```

- 讀取 HDFS 檔案：

```
textFile =
sc.textFile("hdfs://master:9000/user/hduser/wordcount/input/LICENSE.txt")
textFile.count()
```

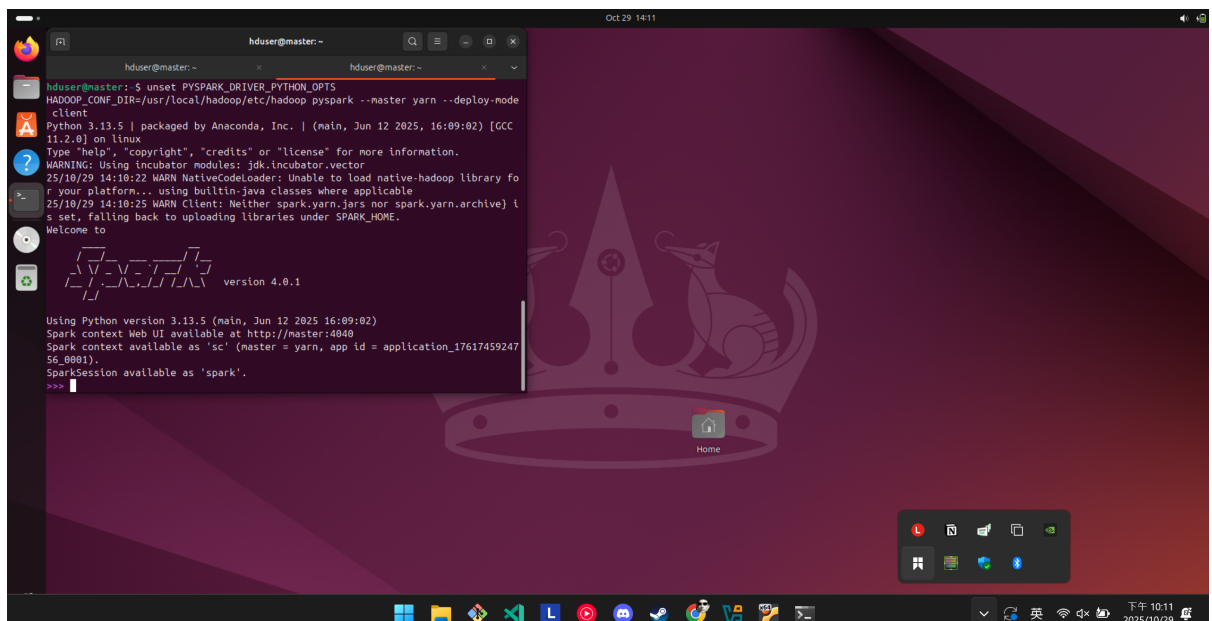
- 離開 pyspark：

```
exit()
```

6. 在 Hadoop YARN 執行 Pyspark

- 啟動：

```
HADOOP_CONF_DIR=/usr/local/hadoop/etc/hadoop pyspark --master yarn --  
deploy-mode client  
sc.master  
textFile =  
sc.textFile("hdfs://master:9000/user/hduser/wordcount/input/LICENSE.txt  
")  
textFile.count()
```



- Spark 運算可從 8088 查看 application 運作。

7. 建置 Spark Standalone Cluster

- 複製 spark-env.sh 樣板並編輯參數：

```
cp /usr/local/spark/conf/spark-env.sh.template  
/usr/local/spark/conf/spark-env.sh  
sudo gedit /usr/local/spark/conf/spark-env.sh  
export SPARK_MASTER_IP=master  
export SPARK_WORKER_CORES=1  
export SPARK_WORKER_MEMORY=512m  
export SPARK_EXECUTOR_INSTANCES=4
```

- 建立 workers 檔案：

```
gedit /usr/local/spark/conf/workers  
# 內容：  
data1
```

```
data2  
data3
```

- 複製 spark 到 data1、2、3，建立目錄設定權限，使用 scp 複製：

```
sudo mkdir /usr/local/spark  
sudo chown hduser:hduser /usr/local/spark  
sudo scp -r /usr/local/spark hduser@data1:/usr/local
```

- 啟動 Spark standalone cluster：

```
/usr/local/spark/sbin/start-all.sh
```

8. Spark Standalone 執行 Pyspark 程式測試

- 啟動：

```
pyspark --master spark://master:7077 --num-executors 1 --total-  
executor-cores 3 --executor-memory 512m  
# 或  
pyspark --master spark://master:7077 --driver-memory 512m --conf  
spark.executor.memory=512m --conf spark.executor.cores=1
```

- 檢查模式與讀取檔案：

```
sc.master  
textFile = sc.textFile("file:/usr/local/spark/README.md")  
textFile.count()  
textFile =  
sc.textFile("hdfs://master:9000/user/hduser/wordcount/input/LICENSE.txt  
")  
textFile.count()
```

- 關閉 Spark：

```
/usr/local/spark/sbin/stop-all.sh
```

- 關閉 Hadoop：

```
stop-all.sh
```

- 關機。

第九章

1. 在 **master**、**data1**、**data2**、**data3** 安裝 **Anaconda**

- 執行安裝腳本：

```
bash Anaconda.XX.XX.sh
```

- 設定 Anaconda 環境變數，編輯 **.bashrc**：

```
sudo gedit ~/.bashrc
```

- 加入以下內容：

```
# ===== Anaconda 3 =====  
export PATH=/home/hduser/anaconda3/bin:$PATH  
export ANACONDA_PATH=/home/hduser/anaconda3  
export PYSARK_DRIVER_PYTHON=$ANACONDA_PATH/bin/jupyter  
export PYSARK_PYTHON=$ANACONDA_PATH/bin/python  
# =====
```

- 重新載入環境變數：

```
source ~/.bashrc
```

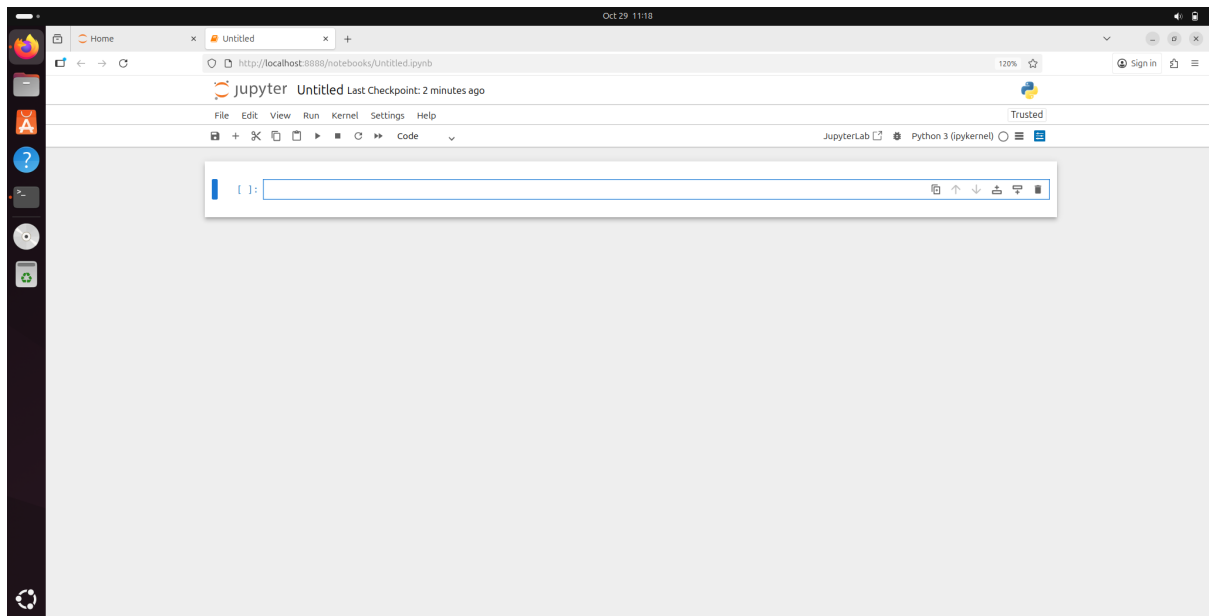
2. 建立 **IPython Notebook** 工作目錄並啟動 **Jupyter**

- 建立工作目錄：

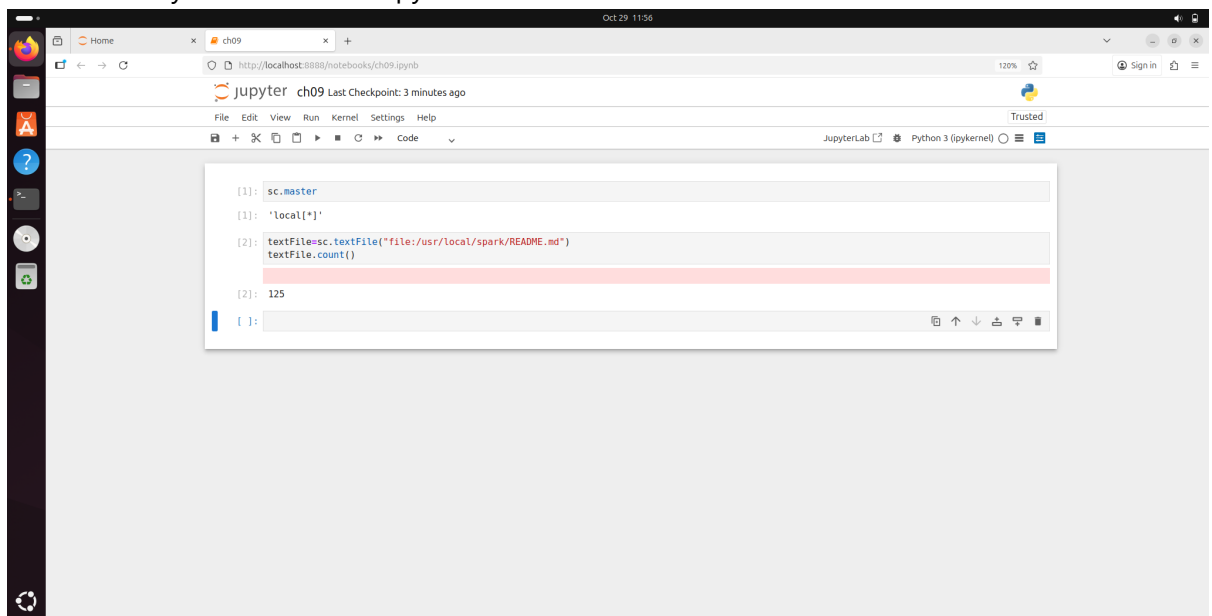
```
mkdir -p ~/pythonwork/ipynotebook  
cd ~/pythonwork/ipynotebook/
```

- 執行 Pyspark 並開啟 Jupyter Notebook：

```
PYSARK_DRIVER_PYTHON=jupyter PYSARK_DRIVER_PYTHON_OPTS="notebook"  
pyspark
```



- 介面示意與 Python 指令可在 Jupyter 中運行。



3. 在 Hadoop Yarn-client 模式下啟用 Jupyter

- 啟動命令：

```
PYSPARK_DRIVER_PYTHON=jupyter PYSPARK_DRIVER_PYTHON_OPTS="notebook"  
HADOOP_CONF_DIR=/usr/local/hadoop/etc/hadoop pyspark --master yarn --  
deploy-mode client
```

```
Oct 29 12:10
hduser@master: ~/pythonwork/ipythonnotebook
hduser@data1: -
hduser@data2: -
hduser@data3: -

Starting datanodes
Starting secondary namenodes [master]
2025-10-29 12:06:07,831 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Starting resourcemanager
Starting nodemanagers
(base) hduser@master: ~/pythonwork/ipythonnotebook$ jps
15474 SecondaryNameNode
15703 ResourceManager
15207 NameNode
10645 Jps
(base) hduser@master: ~/pythonwork/ipythonnotebook$ /usr/local/spark/sbin/stop-all.sh
data3: no org.apache.spark.deploy.worker.Worker to stop
data1: no org.apache.spark.deploy.worker.Worker to stop
data2: no org.apache.spark.deploy.worker.Worker to stop
no org.apache.spark.deploy.master.Master to stop
(base) hduser@master: ~/pythonwork/ipythonnotebook$ /usr/local/spark/sbin/start-all.sh
starting org.apache.spark.deploy.master.Master, logging to /usr/local/spark/logs/spark-hduser-org.apache.spark.deploy.master.Master-1-master.out
data3: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/logs/spark-hduser-org.apache.spark.deploy.worker.Worker-1-data3.out
data2: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/logs/spark-hduser-org.apache.spark.deploy.worker.Worker-1-data2.out
data1: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/logs/spark-hduser-org.apache.spark.deploy.worker.Worker-1-data1.out
(base) hduser@master: ~/pythonwork/ipythonnotebook$ PYSARK_DRIVER_PYTHON=jupyter PYSARK_DRIVER_PYTHON_OPTS="notebook" HADOOP_CONF_DIR=/usr/local/hadoop/etc/hadoop pyspark --master yarn --deploy-mode client
Usage: jupyter [ -h ] [ --version ] [ --config-dir ] [ --data-dir ] [ --runtime-dir ]
      [ --paths ] [ --json ] [ --debug ]
      [ subcommand ]

Jupyter: Interactive Computing

positional arguments:
  subcommand            the subcommand to launch

options:
  -h, --help            show this help message and exit
  --version             show the versions of core Jupyter packages and exit
  --config-dir          show Jupyter config dir
  --data-dir           show Jupyter data dir
  --runtime-dir        show Jupyter runtime dir
  --paths              show all Jupyter paths. Add --json for machine-readable format.
```

```
Oct 29 12:11
Home
ch09
http://localhost:8888/notebooks/ch09.ipynb
120%
jupyter ch09 Last Checkpoint: 18 minutes ago
File Edit View Run Kernel Settings Help
JupyterLab Python 3 (ipykernel)
[3]: sc.master
[3]: 'yarn'
[4]: TextFile=sc.TextFile("file:/usr/local/spark/README.md")
      textFile.count()
[4]: 125
[ ]:
```

- 可在 Hadoop Web UI 監控 Pyspark 的應用程式運行。

Oct 29 12:12

Home ch09 All Applications

http://localhost:8088/cluster/apps

120% Sign in

hadoop All Application

Cluster

About

Nodes

Node Labels

Applications

NEW

NEW SAVING

SUBMITTED

ACCEPTED

RUNNING

FINISHED

FAILED

KILLED

Scheduler

Tools

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Used Resources
1	0	1	0	3	<memory:5 GB, vCores:3>

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes
3	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:1024, vCores:1>

Show 20 entries

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus
application_1761739573595_0001	hduser	PySparkShell	SPARK		default	0	Wed Oct 29 12:08:01 +0000 2025	Wed Oct 29 12:08:03 +0000 2025	N/A	RUNNING	UNDEFINED

Showing 1 to 1 of 1 entries

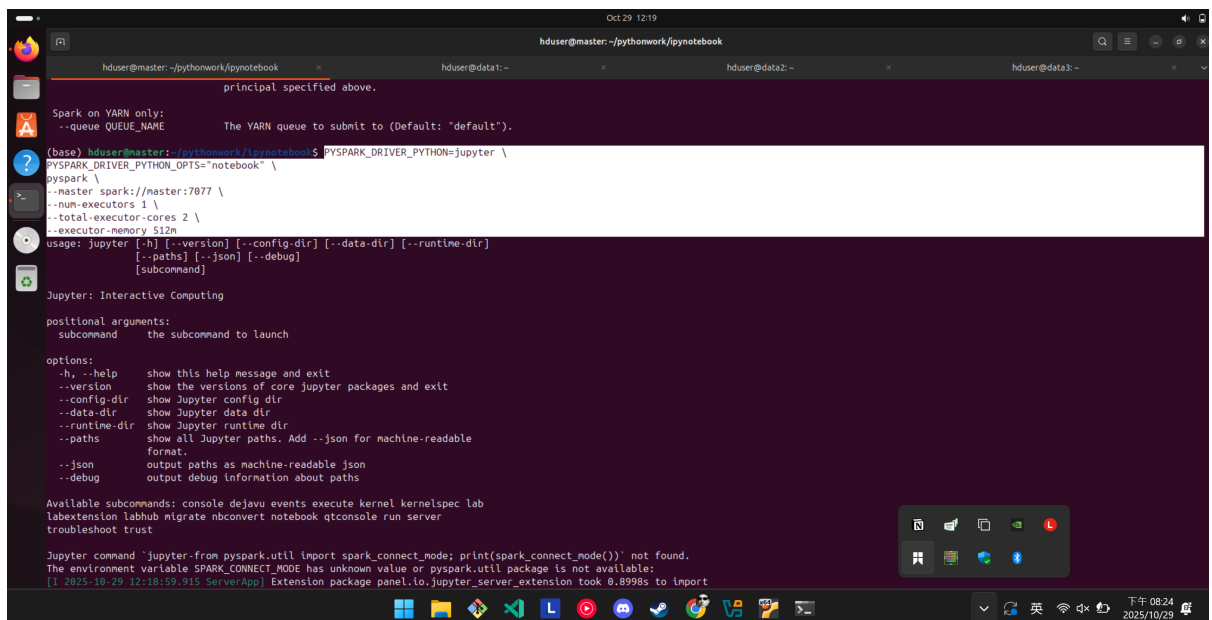
4. Spark Standalone 模式執行 Jupyter Notebook

- 啟動 Spark Standalone cluster :

```
/usr/local/spark/sbin/start-all.sh
```

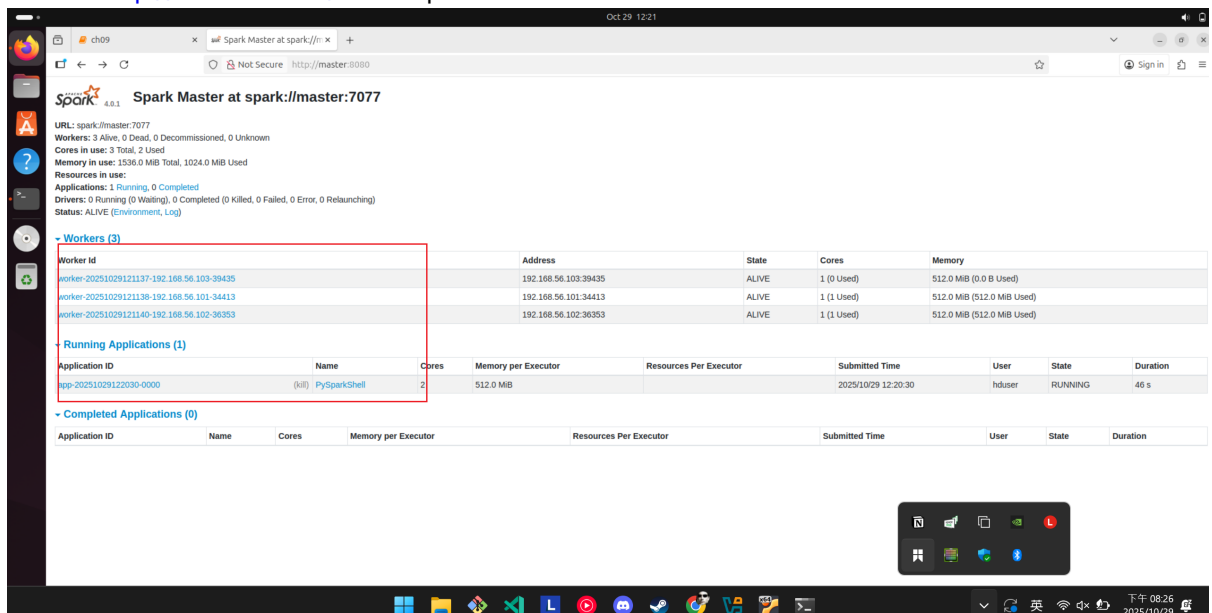
- 執行 Jupyter Notebook 並連接 Spark Standalone :

```
PYSPARK_DRIVER_PYTHON=jupyter \  
PYSPARK_DRIVER_PYTHON_OPTS="notebook" \  
pyspark \  
--master spark://master:7077 \  
--num-executors 1 \  
--total-executor-cores 2 \  
--executor-memory 512m
```



```
Oct 29 12:19  
hduser@master: ~/pythonwork/pynotebook  
principal specified above.  
Spark on YARN only:  
--queue QUEUE_NAME The YARN queue to submit to (Default: "default").  
(base) hduser@master: ~/pythonwork/pynotebook$ PYSPARK_DRIVER_PYTHON=jupyter \  
PYSPARK_DRIVER_PYTHON_OPTS="notebook" \  
pyspark \  
--master spark://master:7077 \  
--num-executors 1 \  
--total-executor-cores 2 \  
--executor-memory 512m  
usage: Jupyter [ahi] [-v|--version] [--config-dir] [--data-dir] [--runtime-dir]  
[...paths] [--json] [--debug]  
[subcommand]  
Jupyter: Interactive Computing  
positional arguments:  
subcommand the subcommand to launch  
options:  
-h, --help show this help message and exit  
-v, --version show the versions of core jupyter packages and exit  
--config-dir show Jupyter config dir  
--data-dir show Jupyter data dir  
--runtime-dir show Jupyter runtime dir  
--paths show all Jupyter paths. Add --json for machine-readable  
format.  
--json output paths as machine-readable json  
--debug output debug information about paths  
Available subcommands: console dejavu events execute kernel kernelspec lab  
labextension labhub migrate nbconvert notebook qtconsole run server  
troubleshoot trust  
Jupyter command 'jupyter-from-pyspark.util import spark_connect_mode; print(spark_connect_mode())' not found.  
The environment variable SPARK_CONNECT_MODE has unknown value or pyspark.util package is not available:  
[1 2025-10-29 12:18:59.915 ServerApp] Extension package panel.to.jupyter_server_extension took 0.8998s to import
```

- 可透過 <http://master:8080/> 監控 Spark Web UI ◦



Worker ID	Address	State	Cores	Memory
worker-20251029121137-192.168.56.103-39435	192.168.56.103:39435	ALIVE	1 (0 Used)	512.0 MB (0.0 B Used)
worker-20251029121139-192.168.56.101-34413	192.168.56.101:34413	ALIVE	1 (1 Used)	512.0 MB (512.0 MB Used)
worker-20251029121140-192.168.56.102-36353	192.168.56.102:36353	ALIVE	1 (1 Used)	512.0 MB (512.0 MB Used)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20251029122030-0000	(kill) PySparkShell	2	512.0 MB		2025/10/29 12:20:30	hduser	RUNNING	46 s